Web Applications – Exam #3 (deadline 2026-09-03 at 23:59)

"Sport"

FINAL version - Modifications highlighted with RED text.

Design and implement a web application to reserve facilities in a sport center. Note that, for simplicity, dates must not be handled, thus a facility is either booked or not, for an unspecified date and amount of time.

Facilities must be identified by a unique string decided by the student, e.g., T3 is the Tennis court #3. Facilities are: 3 tennis courts, 2 basketball courts, 2 volleyball courts, 1 soccer field, 4 table tennis tables, 2 cycling tracks.

Moreover, the sports center also provides equipment for rental. For every facility, some equipment type with a specified minimum quantity (value in parenthesis) is mandatory. Equipment types without a minimum are optional.

- Tennis court: tennis racket (2), tennis balls (3), towels;
- Basketball court: basketball (1), cones;
- Volleyball court: volleyball (1), pair of knee pads;
- Soccer field: soccer ball (1), pair of soccer shoes (10), pair of goalkeeper gloves;
- Table tennis table: table tennis rackets (2), table tennis balls (1);
- Cycling track: bicycle (1), helmet (1), repair kit.

Equipment for rental is limited across the whole sport center, and includes 8 tennis rackets, 7 tennis balls, 4 towels, 2 basketballs, 4 cones, 2 volleyballs, 10 pairs of knee pads, 2 soccer balls, 12 pairs of soccer shoes, 2 pairs of goalkeeper gloves, 8 table tennis rackets, 4 table tennis balls, 4 bicycles, 4 helmets, 1 repair kit. Equipment cannot be rented without renting the corresponding facility.

Anybody accessing the first page of the application must be able to visualize the number of available facilities of each type and the quantity currently available for each equipment type. Any user, at login time, may authenticate either with username/password only or with username/password plus TOTP, to be requested in a separate second screen after username/password verification. The role of TOTP login is clarified later in the text.

Authenticated users can book facilities together with equipment according to their score (detailed later). Once such booking is confirmed from the server, it becomes a reservation (and all the views in the application must be updated accordingly). Each reservation includes one facility and the corresponding mandatory minimum quantities of all required equipment types (Example: 1 tennis court with 2 tennis rackets and 3 tennis balls). Additional quantities of mandatory or optional equipment may also be requested (Example: 1 tennis court with 3 tennis rackets, 5 tennis balls, 1 towel). Renting the equipment is mandatory since customers cannot use their own equipment. When it is known that not enough equipment is available to satisfy the request, booking the corresponding facility must not be allowed. Moreover, the equipment quantities that can be requested are also limited by the user score, as detailed later.

Facility selection can be done using two different mechanisms: direct selection from a list or automatic assignment by the system, among the type of facility chosen by the user (e.g., tennis court, soccer field, etc.) Equipment must always be selected by the user. Each user can reserve more than one facility, also if they are of the same type.

The design of the user interface is up to the student. It is expected that all choices are made carefully by considering security and reasonable usability. However, the following specifications must be satisfied. It must be possible to:

- see the list of reservations, including equipment, currently reserved for the user.
- modify already confirmed reservations by adding and/or removing rented equipment (except mandatory ones);
- delete a reservation.

Deleting a reservation restores facility and equipment availability. Each user is also associated with a personal score, that is an integer number less than or equal to zero. Users who did not perform any operation have a personal score equal to zero. The user's score is reduced by 1 for each "delete reservation" operation. Users with a negative score may create reservations, but only with the minimum required equipment quantities. Existing reservations may still be modified but only to remove equipment. They may not request optional or additional mandatory equipment. The negative score goes back to zero if the user, at next login, opts for the 2FA procedure with a TOTP. In case the user is already logged in, the user must logout and login again to reset the score.

Finally, the application should prevent a user from re-booking, within 30 seconds, any facility of the same type (e.g, all tennis courts) that the same user had previously reserved and then released. In all cases, after every reservation creation, modification, or deletion request, the interface must show whether the operation succeeded or failed and, in case of failure, the reason, e.g., not enough facilities, not enough equipment of type X, too early to reserve again, etc.

The organization of these specifications into different screens (and potentially different routes), when not specified, is left to the student and is subject to evaluation.

For the 2FA procedure, for simplicity, use the following secret for all users: `LXBSMDTMSP2I5XFXIYRGFVWSFI` (it is the same used during lectures and labs).

NB: Very important: using the same secret for all users is just for simplicity, the secret has to be stored separately, one for each user that requires 2FA, in the database (as seen during lecture examples).

Project requirements

- User authentication (login and logout) and API access must be implemented with `passport.js` and **session <u>cookies</u>, using the mechanisms explained during the lectures**. Session information must be stored on the server, as done during the lectures. The credentials must be stored in hashed and salted form, as done during the lectures. **<u>Failure to follow this pattern will automatically lead to exam failure.</u>**
- The communication between client and server must follow the multiple-server pattern, by properly configuring CORS, and React must run in "development" mode with Strict Mode activated. **<u>Failure to follow this pattern will automatically lead to exam failure.</u>**

- The project must be implemented as a **React application** that interacts with an HTTP API implemented in Node+Express. The database must be stored in an SQLite file. **Failure to follow this pattern will automatically lead to exam failure.**
- 2FA verification must be implemented using a TOTP and by using the libraries explained during the lectures, and with the secret specified in the exam text. **Failure to follow this pattern will automatically lead to exam failure.**
- The evaluation of the project will be carried out by navigating the application, **using the starting URL http://localhost:5173**. Neither the behavior of the "refresh" button, nor the manual entering of a URL (except /) will be tested, and their behavior is not specified. Also, the application should never "reload" itself as a consequence of normal user operations.
- The user registration procedure is not requested *unless specifically required in the text*.
- The application architecture and source code must be developed by adopting the best practices in software development, in particular those relevant to single-page applications (SPA) using React and HTTP APIs.
- Particular attention must be paid to ensure server APIs are duly protected from performing unwanted, inconsistent, and unauthorized operations.
- The **root directory** of the project **must contain** a README.md file and have the same subdirectories as the template project (`client` and `server`). Additional subdirectories can be created in `server` and `client/src` as needed. The project must start by running these commands: "`cd server; nodemon index.mjs`" and "`cd client; npm run dev`". A template for the project directories is already available in the exam assignment page. **Do not move or rename such directories**. You may assume that nodemon is globally installed. **Failure to follow this pattern will have a heavy impact on the final grade.**
- The whole project must be submitted according to the course instruction.
- The project **must not include** the `node_modules` directories. They will be re-created by running the "`npm ci`" command.
- The project **must include** the `package-lock.json` files for each of the folders (client and server).
- The project may use popular and commonly adopted libraries (for example `dayjs`, `react-bootstrap`, etc.), if applicable and useful. Such libraries must be correctly declared in the `package.json` and `package-lock.json` files, so that the `npm ci` command can download and install all of them.

Database requirements

- The database schema must be designed and decided by the student, and it is part of the evaluation..
- The database must be implemented and populated by the student. It must be preloaded with all facilities and equipment. There must be at least four users: one user without reservations, two users with one reservation each, and one user with two reservations. After the preloaded reservations have been created, at least one unit of each equipment type must remain available. In addition, at least one facility of each type must remain available. Two users must have a negative score.

Quality requirements

In addition to the implementation of the required application functionality, the following quality requirements will be evaluated, as usual:

- Database design and organization

- Design of the HTTP APIs.
- Organization of React components and routes.
- Correct usage of React patterns (functional behavior, hooks, state, context, and effects).
- Avoiding direct manipulation of the DOM or browser behaviors/functionalities outside React (including but not limited to: alert, confirm, etc.).
- Code clarity.
- Absence of errors (and warnings) in the client console (except those caused by errors in the imported libraries and by HTTP status different from 2xx).
- Absence of application crashes or unhandled exceptions.
- Basic usability and user-friendliness.
- Originality of the solution.

Contents of the README.md file

The README.md file must contain the following information (an example is available in the template project to be used for submission). The clarity and correctness of this file is also part of the evaluation. Generally, each information should take no more than 1-2 lines.

1. Server-side:
    a. A list of the HTTP APIs offered by the server, with a short description of the parameters and of the exchanged objects.
    b. A list of the database tables, with their purpose, and the name of the columns.
2. Client-side:
    a. A list of 'routes' for the React application, with a short description of the purpose of each route.
    b. A list of the main React components developed for the project. Minor ones can be skipped.
3. Overall:
    a. A screenshot of, at least, the **facility selection page on the map**. The screenshot must be embedded in the README by linking the image committed in the repository.
    b. Usernames and passwords of all the users in the database, including their characteristics (how many reservations for each one, initial score, etc.) according to the database section.

Submission procedure (IMPORTANT)

NOTE: CHANGED w.r.t. previous exams due to github classroom being discontinued

To correctly submit the project, you must:

- **Be enrolled** in the exam call (mandatory!).
- Upload a zip file containing the project, with the structure exactly as provided in the template on the course website here: https://github.com/polito-WA-2026/.github/blob/main/profile/exam-template.zip , on the didattica.polito.it course webpage in the "Homework" tab, BEFORE THE DEADLINE. Note that the course webpage allows submissions at any time, but late submissions will not be considered.

- If you already submitted the zip file and you wish not to be considered in the exam, simply un-enroll from the exam call from the didattica.polito.it webportal. This can be done at any time before the exam.

To test your submission, these are the exact commands that the teachers will use to download and run the project. You may wish to test them in an empty directory:

```
unzip ...yourProjectZipFile...
(cd client ; npm ci; npm run dev)
(cd server ; npm ci; nodemon index.mjs)
```

Make sure that all the needed packages are downloaded by the `npm ci` commands. Be careful: if some packages are installed globally, on your computer, they might not be listed as dependencies. Always check it in a clean installation (for instance, in an empty Virtual Machine).

The project will be **tested under Linux**: be aware that Linux is **case-sensitive for file names**, while Windows and macOS are not. Double-check the upper/lowercase characters, especially of `import` and `require()` statements, and of any filename to be loaded by the application (e.g., the database).